E-Commerce Microservices on OKE: Architecture and Implementation

Documentation	Final Project Documentation
Team	Group A
Compartment	`shared-group-a-cmp`
Region	`me-jeddah-1` (Saudi Arabia West – Jeddah)
Repositories	`github.com/Ejada-A`

Team Members

Basel Alaa	Ali Youssef
Yara Adel	Chris Mina Kamal
Abdelrahman Darwish	Habiba Fawzy
Ali Ahmed Fakhry Hamad	Mohamed Kholosy
Ali Tallawy	Peter Kameel
Salma Azara	Beshoy Sorial

Technical Summary

In this project, we split an e-commerce application into four independently deployable microservices (`auth-service`, `products-service`, `orders-service`, and `payments-service`) and a Next.js frontend. We deployed the system to Oracle Kubernetes Engine (OKE) through a Terraform-provisioned cluster and a Helm/ArgoCD GitOps pipeline. Moreover, we mapped each service to its own Git repository and container image so that a push to `main` can reach a running pod without a manual `kubectl apply` step.

- **We separated the application into four service-owned data models.** However, we found that the services still share one database: `orders-service` and `payments-service` read and write the `products` collection directly rather than calling `products-service` over HTTP; the backend services do not call each other.
- **We implemented a fully GitOps-driven infrastructure workflow.** Each service's CI pipeline builds and pushes its image to OCIR, a shared workflow updates the matching tag in the Helm chart's `values.yaml`, and ArgoCD reconciles the cluster automatically.
- **We used production QA to drive a focused security-remediation cycle.** We closed all six critical findings, including unauthorized mutations, an exposed database-seed route, invalid order quantities, and unverified Stripe payment confirmation (Section 4.9).

Not everything landed: three storefront pages and the admin dashboard's metrics remain placeholders. We also created a separate QA repository that now provides production regression coverage, with 93 of 95 checks passing after remediation, though we have not yet integrated the suite into the six implementation repositories. We discuss this and our other next steps in Chapter 6.

Contents

1 Project Overview

For the final week of the Ejada Cloud Build Track internship, our Group A team split an e-commerce application into five independently deployable Git repositories: four backend services and a Next.js frontend. We then deployed the system to Oracle Kubernetes Engine (OKE) using a Terraform-provisioned cluster and a Helm/ArgoCD GitOps pipeline.

1.1 Objectives and Scope

We set two main objectives for the build: first, to split the application into microservice boundaries based on data ownership, each with its own repository and container image; and second, to automate the path from a code push to a running pod.

For our initial scope, we focused on: one OCI region (`me-jeddah-1`), one compartment (`shared-group-a-cmp`), one OKE cluster and namespace (`ecommerce-ns`), the five application repositories above, and the `Terraform-code` repository holding the infrastructure, Kubernetes manifests, and Helm chart.

We initially treated as out of scope: automated testing, multi-region or multi-availability-domain high availability, and a production security review. However, we later added a dedicated QA and remediation cycle (Section 4.9). We also found that three customer-facing storefront pages remain incomplete (Section 5.4) and that a few areas of our implementation differ from the original design draft, as we summarise in Section 2.7.

1.2 Project Ownership

We summarise each team member's area of responsibility in Table 1.1.

Table 1.1: Team members and areas of responsibility.

Member	Area	Member	Area
Ali Ahmed Fakhry Hamad	Deployment	Mohamed Kholosy	E-commerce application
Abdelrahman Darwish	E-commerce application	Habiba Fawzy	Database
Salma Azara	Terraform code	Chris Mina Kamal	Terraform code
Ali Tallawy	Testing	Basel Alaa	Terraform code
Beshoy Sorial	CI/CD	Yara Adel	Architecture and design
Ali Youssef	Documentation	Peter Kameel	Testing

2 System Architecture

2.1 Repository Overview

We organized the system across six repositories under the `Ejada-A` GitHub organization. Table 2.1 summarises the responsibility and structure of each one.

Table 2.1: Repositories and responsibilities.

Repository	Contents
`auth-service`	One entry point, `server.ts` (206 lines), plus `user.model.ts` and `admin.model.ts`. There are no separate routes/controllers/services directories – route handlers live directly in `server.ts`.
`products-service`	Same shape: `server.ts` (191 lines), `product.model.ts`, `category.model.ts`.
`orders-service`	`server.ts` (128 lines), `order.model.ts`, and its own copy of `product.model.ts`.
`payments-service`	`server.ts` (98 lines), its own copies of `order.model.ts` and `product.model.ts`.
`ecomm-ui`	A Next.js 15 (App Router) application: storefront pages, an `/admin` section, and a set of `/api/*` routes that act as a backend-for-frontend (BFF), proxying to the four backend services over HTTP.
`Terraform-code`	The Terraform `network` and `oke` modules, the Helm chart (`helm/ecommerce-app`), a parallel set of raw Kubernetes manifests (`kubernetes/`), ArgoCD Application/Ingress definitions (`argocd/`), and the deployment runbook `DEPLOYMENT.md`.

We found that each backend service uses a flat Express structure. Routes and request handlers are defined in `server.ts`, where each handler calls the corresponding Mongoose model directly; there is no separate controller, service, or repository layer.

Note

We maintain our production QA coverage in the dedicated `EjadaStore-Testing` repository (Section 4.9). However, we found that the six implementation repositories do not yet include this suite in their own CI workflows; `Terraform-code`'s `test-runner.yml` contains a placeholder check (`echo "Runner works"`).

2.2 Service Responsibilities

We divided the backend into four services, each with its own repository: `auth-service`, `products-service`, `orders-service`, and `payments-service`. Table 2.2 sets out what each service owns.

Table 2.2: Service responsibilities.

Service	Owns
`auth-service`	User accounts and admin accounts (separate Mongoose models), credential hashing, admin login tokens
`products-service`	Product catalog and categories; also owns the one-time database seed (pulled from a public demo API, `api.escuelajs.co`)
`orders-service`	Orders and order line items; validates stock and price and decrements stock at order-creation time
`payments-service`	Stripe Checkout session creation; also creates its own `Order` record for the checkout flow

Service Communication

During our review, we found that the backend services do not call each other over HTTP. Instead, `orders-service` and `payments-service` each declare their own local copy of the `product.model.ts` Mongoose schema and query it directly, since all five workloads connect to the same MongoDB instance and database; Mongoose resolves a model named `Product` to the same underlying `products` collection regardless of which service defines the schema. The only HTTP orchestration in the system happens in `ecomm-ui`'s BFF routes, which call each backend service by its Kubernetes Service DNS name.

2.3 Technology Stack

We summarise the technologies and versions used across the application and infrastructure layers.

Table 2.3: Technology stack.

Layer	Notes
Backend runtime	Node.js 20 (`node:20-slim`), TypeScript run directly via `tsx` (no separate compile step in `start`)
Backend framework	Express 4.19, `cors`, `dotenv` – identical across all four backend services
Backend data access	Mongoose 8.9 against a single shared MongoDB 7.0 instance
Auth-specific	`bcryptjs` 2.4 for password hashing; `jose` 6.2 (`SignJWT`, HS256) for admin JWTs only
Payments-specific	`stripe` 17.5 (Node SDK), Stripe Checkout Sessions
Frontend	Next.js 15.5 (App Router), React 19, Tailwind CSS 3.4, `lucide-react` icons
Frontend state	React Context + `localStorage` for the cart (`CartContext.tsx`); a second Zustand store also exists (`hooks/useCart.ts`) – see Section 5.4
Frontend dependencies declared but unused	`zod`, `next-auth`, `bcryptjs` (declared in `package.json`, not currently imported by application code)
Container orchestration	Kubernetes v1.34.2 (OKE-managed), Helm 3 chart `ecommerce-app`
Infrastructure as code	Terraform, provider `oracle/oci` $\geq$ 5.30, provider `integrations/github` $\sim$ 6.0; remote state in an S3-compatible OCI Object Storage backend
GitOps	ArgoCD (upstream manifests, installed by `deploy.yml`), NGINX Ingress Controller

All five workloads share a single MongoDB instance with no per-service database split.

2.4 Deployed Architecture

To present the deployed design clearly, we use Figure 2.1 to show the network and OKE architecture: four subnets, the Internet/NAT/Service gateway split, and the shared MongoDB PVC in the Pods subnet.

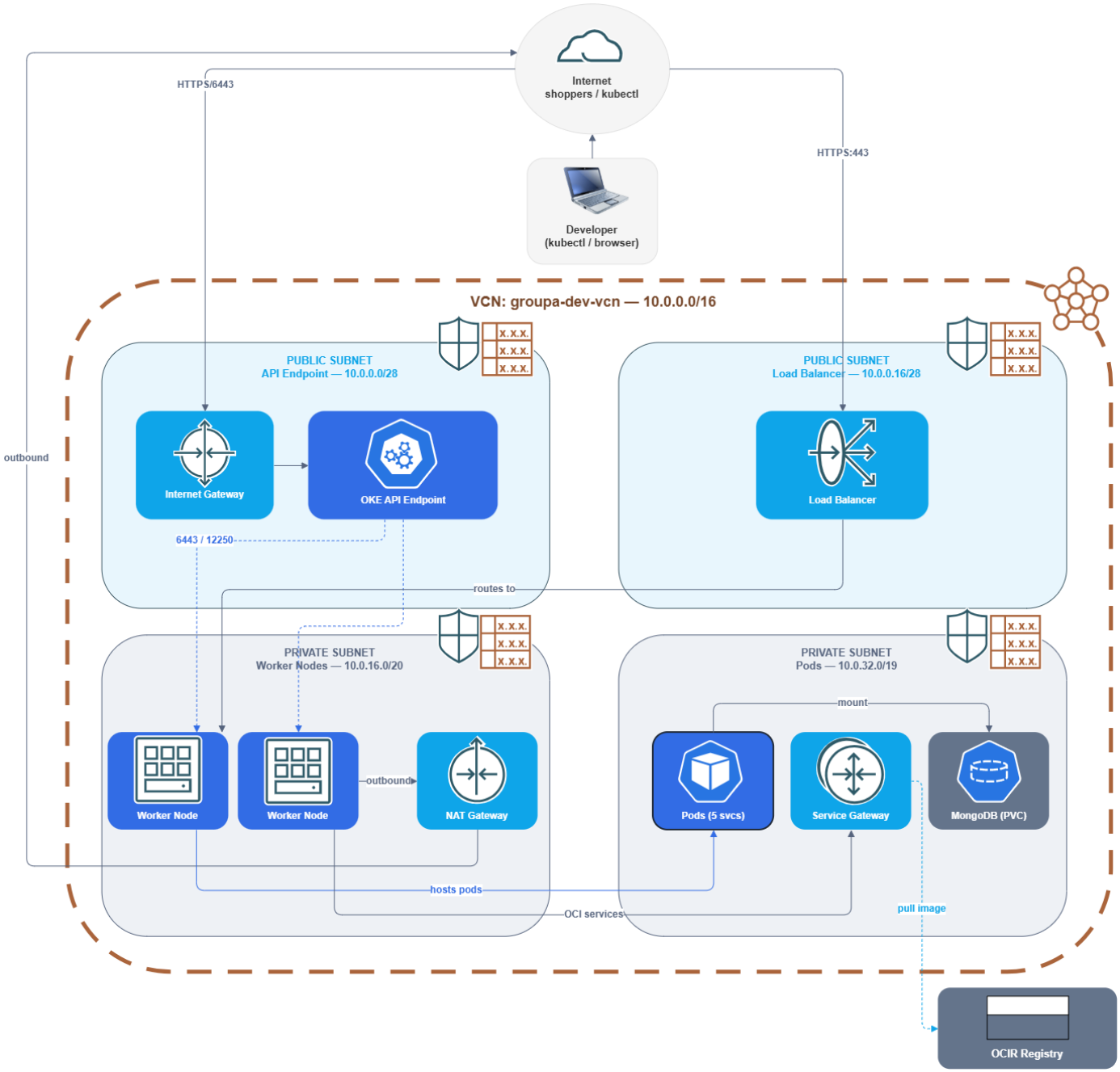

Figure 2.1: Deployed architecture inside VCN `groupa-dev-vcn` (`10.0.0.0/16`).

Neither the API endpoint path nor the Load Balancer path currently restricts inbound traffic by source IP; Chapter 3 covers the network configuration in full.

Moreover, Figure 2.2 shows the application-layer routing: the ingress routes by path to each of the five workloads, all of which connect to the same MongoDB instance.

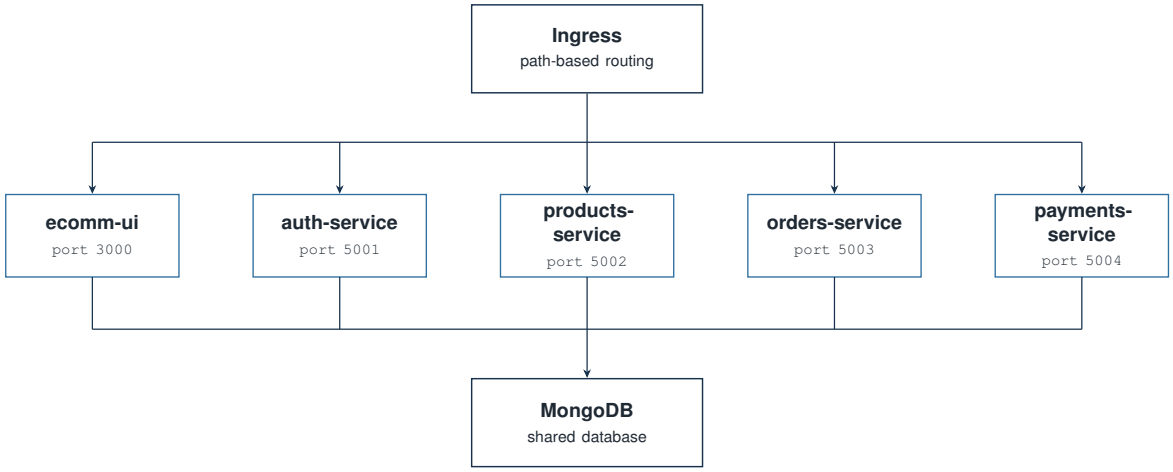

Figure 2.2: Application-layer access structure.

Note

`ecomm-ui` is not a pure stateless proxy in front of the backend services. Most of its `/api/*` routes forward requests to the corresponding backend service. The frontend's actual data path is HTTP calls to the four backend services.

`Terraform-code` includes two deployment definitions for the same application: the Helm chart (`helm/ecommerce-app`) and a parallel set of raw manifests (`kubernetes/00` through `06`). ArgoCD's `Application` resource points at `helm/ecommerce-app`, so the Helm chart is the path GitOps manages; the raw manifests are a second, manually applied option documented in `DEPLOYMENT.md` as "Option B" and are not kept in sync with the Helm chart automatically. The two paths currently define different autoscaling policies (Section 5.3).

2.5 Network Architecture

We configured the OKE cluster network with four subnets, which we summarise in Table 2.4. The full set of gateway, route table, and NSG rules is in Appendix B.

Table 2.4: OKE subnet configuration.

Subnet	Type	Purpose	CIDR	Range
API Endpoint	Public	Kubernetes API access	10.0.0.0/28	.0–.15
Load Balancer	Public	Public entry point for the application	10.0.0.16/28	.16–.31
Worker Nodes	Private	Hosts the VMs running all pods	10.0.16.0/20	10.0.16.0–10.0.31.255
Pods	Private	IP addresses for individual pods (VCN-native)	10.0.32.0/19	10.0.32.0–10.0.63.255

Note

We found that the network module (`modules/network/variables.tf`) defines four boolean switches, each defaulting to `false`, that control worker SSH access, general worker outbound traffic, pod outbound internet access, and public NodePort access. The root module (`main.tf`) sets all four to `true`. Chapter 3 lists exactly what this opens and covers the security implications.

2.6 Scaling and Routing

We configured every workload to scale through its own Horizontal Pod Autoscaler, while the ingress routes to each service by path. Moreover, Section 5.3 covers the exact policy and a difference between the two deployment paths.

2.7 Design Draft vs. Implementation

We began with the Architecture & Design Draft as our intended design. However, our source review showed that some implementation decisions evolved during the build. Table 2.5 summarises these differences, including areas the draft did not originally cover.

Table 2.5: Design draft vs. current implementation.

Area	Classification	Current implementation
Domain-to-service mapping	Implemented	Four domains map 1:1 to four repositories, as designed
Inter-service communication	Implemented differently	The original design anticipated service-to-service API calls; the implemented services instead access shared MongoDB collections directly (Section 2.2)
NSG-restricted API/SSH access	Implemented differently	The root Terraform configuration enables all four optional network access switches and allows public access to the Kubernetes API endpoint (Section 2.5)
Secrets via OCI Vault	Planned	Referenced in `DEPLOYMENT.md` but not implemented in the current Terraform configuration or GitHub Actions workflows (Chapter 3)
Role-based product/order authorization	Planned	Backend services and BFF routes do not currently check a JWT or role before mutating data (Chapter 3)
User authentication	Partially implemented	Credentials are validated server-side, but no session or token is currently issued to a regular user (Chapter 3)
Payment confirmation	Partially implemented	A Stripe Checkout session is created server-side; order status is confirmed through a client call rather than a webhook or session check (Chapter 3)
Full storefront browsing	Partially implemented	The homepage and admin panels are functional; the product listing, product detail, and category pages remain incomplete (Section 5.4)
Two parallel deployment manifests	Repository-only	A raw-manifest path and a Helm-chart path both exist and currently define different HPA policies; the draft describes neither (Section 5.3)
TLS certificate source	Implemented differently	An AWS ACM certificate was tried first for the ingress; ACM certificates are locked to AWS infrastructure and cannot export a private key, so the team moved to cert-manager with a Let's Encrypt `ClusterIssuer` instead (Section 4.5)
Automated testing	Implemented separately	We maintain the production regression suite in `EjadaStore-Testing`; CI integration with the six implementation repositories remains a next step

3 Security Considerations

In this chapter, we record the security baseline we observed at the source commits listed in Chapter 5. We cover network access, authentication and authorization, payment confirmation, secrets management, IAM, the data layer, and pod-level hardening. However, this baseline predates our later production remediation and retest in Section 4.9; where the two differ, we use the retest as the newer operational evidence.

3.1 Network-Level Access

We summarise the network paths we found open to the cluster, based on `modules/network/locals.tf` and the boolean overrides in `main.tf` (Section 2.5).

Table 3.1: Network access rules.

Path	Current rule
Kubernetes API endpoint	TCP 6443 open to `0.0.0.0/0`, not restricted to team IPs
Load Balancer	TCP 80 and TCP 443 both open to `0.0.0.0/0`
Worker node SSH	TCP 22 open to `0.0.0.0/0` (`enable_worker_ssh_access = true` in `main.tf`)
Worker node NodePort range	TCP 30000–32767 open to `0.0.0.0/0`, bypassing the Load Balancer (`enable_nodeport_public_access = true`)
Worker node general outbound	Unrestricted outbound TCP to `0.0.0.0/0` (`enable_worker_general_outbound = true`)
Pod outbound	TCP 443 to `0.0.0.0/0` (`enable_pods_outbound_internet = true`)
ArgoCD server UI	Exposed via a public NGINX ingress (`argocd/argocd-ingress.yaml`), HTTPS force-redirected

We found that the network module's optional access switches default to off; however, the root module enables all of them. Worker nodes and pods still reach OCI services (including OCIR image pulls) through the private Service Gateway rather than the NAT path, matching the route tables in Appendix B.

3.2 Authentication and Authorization

Backend Authorization at the Initial Review

In our initial review, we found that the backend services (`auth-service`, `products-service`, `orders-service`, `payments-service`) do not check a token, header, or role on any route. `products-service`'s `POST`, `PUT`, and `DELETE /products` routes – and the `ecomm-ui` BFF routes and Server Actions that call them – accept requests from any caller that can reach the service.

We found two authentication paths in the system:

- **Admin authentication.** `auth-service`'s `POST /auth/admin/login` issues a `jose`-signed HS256 JWT (1-day expiry) after checking a `bcryptjs` password hash. `ecomm-ui` stores the token

in an httpOnly `admin_token` cookie (`/api/admin/login/route.ts`), and `middleware.ts` verifies that cookie for requests to `/admin/*` pages. The underlying API routes those pages call are not covered by this check.

- **Standard user login.** `auth-service`'s `POST /auth/user/login` checks the password hash and returns user data but does not issue a token. `ecomm-ui`'s login page notes this directly in a comment: `// In a real app, save token/session here`. No user-scoped session exists after a successful login.

3.3 Payment Confirmation

We found that `payments-service` creates a Stripe Checkout Session and a `pending Order` record. When Stripe redirects the browser back to `/checkout/success`, that page calls `POST /api/orders/confirm` with the `orderId` and `sessionId` from the URL. The route does not call Stripe to verify the session before updating the order; it sets `status: "paid"` directly. The frontend's own source acknowledges this:

Listing 3.1: `src/app/(store)/checkout/success/page.tsx`

```
// In a real production app, you would verify the session status
// with your backend here, or use a Stripe Webhook to mark the order paid.
// For this tutorial, we will trust the redirect.
```

We also found no Stripe webhook handler in `payments-service` or elsewhere in the repositories at the reviewed commits. Our subsequent remediation added Stripe session validation: the production retest rejected a fabricated session ID and closed `BUG-018` (Section 4.9).

3.4 Secrets Management

We found two different secrets-management patterns across the repository:

- **Automated deployment (`deploy.yml`).** `JWT_SECRET`, `STRIPE_SECRET_KEY`, `ADMIN_EMAIL`, and `ADMIN_PASSWORD` are read from encrypted GitHub Actions secrets and written into a Kubernetes Secret (`app-secrets`) at deploy time. The same `ADMIN_EMAIL` secret is also passed to cert-manager's `letsencrypt-prod ClusterIssuer` as the ACME registration email (Section 4.5). `DEPLOYMENT.md` documents an equivalent manual path using a gitignored `.env` file. Neither commits real secret values to Git.
- **Plaintext values in `values.yaml`.** `helm/ecommerce-app/values.yaml` also contains a plaintext `secrets:` block with a JWT secret, a Stripe secret key in live key format, and a default admin email/password. No Helm template references `.Values.secrets`, so the block does not affect what gets deployed. It should still be removed and the associated credentials rotated, since it remains in the repository's Git history.

Moreover, we found that OCI Vault integration is referenced in `DEPLOYMENT.md`, which lists three GitHub secret names implying Vault-stored OCIDs (`VAULT_OCIR_AUTH_TOKEN_OCID`, `VAULT_STRIPE_SECRET_KEY_OCID`, `VAULT_JWT_SECRET_OCID`). None of these names appear in any Terraform file or GitHub Actions workflow, and no Vault resource exists in `Terraform-code`; secrets are managed through Kubernetes Secrets instead.

3.5 IAM

In our IAM review, we found that `main.tf` contains a commented-out dynamic group and IAM policy for OKE worker access to images and secrets, with an inline note that the team did not have tenancy-level access to create dynamic groups. The current deployment uses a static, permanent OCI Auth Token (`ocir-secret`, a Kubernetes `docker-registry` Secret) instead. An earlier version generated this token from the OCI CLI's short-lived `access-token get` command, which expired and took down image pulls cluster-wide with `ImagePullBackOff` once it did; the permanent token avoids that failure mode.

3.6 Data Layer

We observed that MongoDB is deployed as a single replica (`helm/ecommerce-app/templates/mongodb.yaml`) with no authentication configured. Any pod that can reach the `mongodb` ClusterIP Service in the namespace can read or write every collection without credentials, and the single-replica deployment is a shared dependency for all five workloads with no built-in redundancy.

3.7 Pod-Level Hardening

However, we also confirmed a consistent pod-hardening measure: every workload's Deployment sets `securityContext: runAsNonRoot: true` with an explicit non-root UID (1000 for the four backend services, 1001 for `ecomm-ui`), matching each Dockerfile's own `USER` directive, applied consistently across all five workloads.

4 Infrastructure and Deployment

In this chapter, we explain how we provisioned the OKE cluster, how our code changes reach running pods, and how we deploy and configure the system.

4.1 Terraform Module Structure

We structured the root module (`Terraform-code/main.tf`) around two child modules, `network` and `oke`, resolves the latest Oracle Linux 8 image for the chosen compute shape, and provisions one OCIR repository per microservice from a single `for_each` over `local.microservices`. State is stored in an S3-compatible OCI Object Storage backend (`providers.tf`), not locally. The cluster runs Kubernetes v1.34.2, set explicitly in the `oke` module call.

Module Interfaces

We use Figure 4.1 to show the `network` module's inputs and outputs, including four boolean access switches that default to `false` in the module itself (Section 2.5).

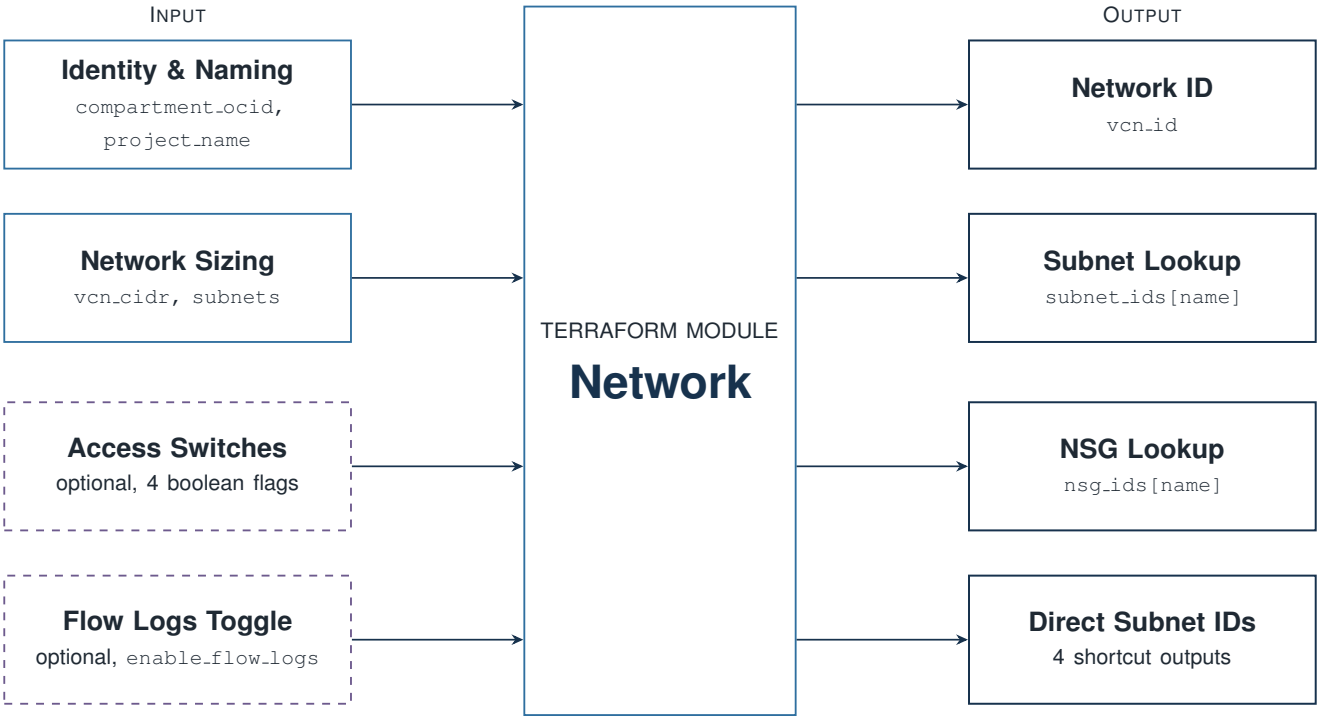

Figure 4.1: Terraform `network` module inputs and outputs. Solid nodes are required inputs and outputs; dashed nodes are the optional access and logging switches.

Moreover, Figure 4.2 shows the `oke` module's interface; network placement and NSG IDs are consumed directly from the `network` module's outputs.

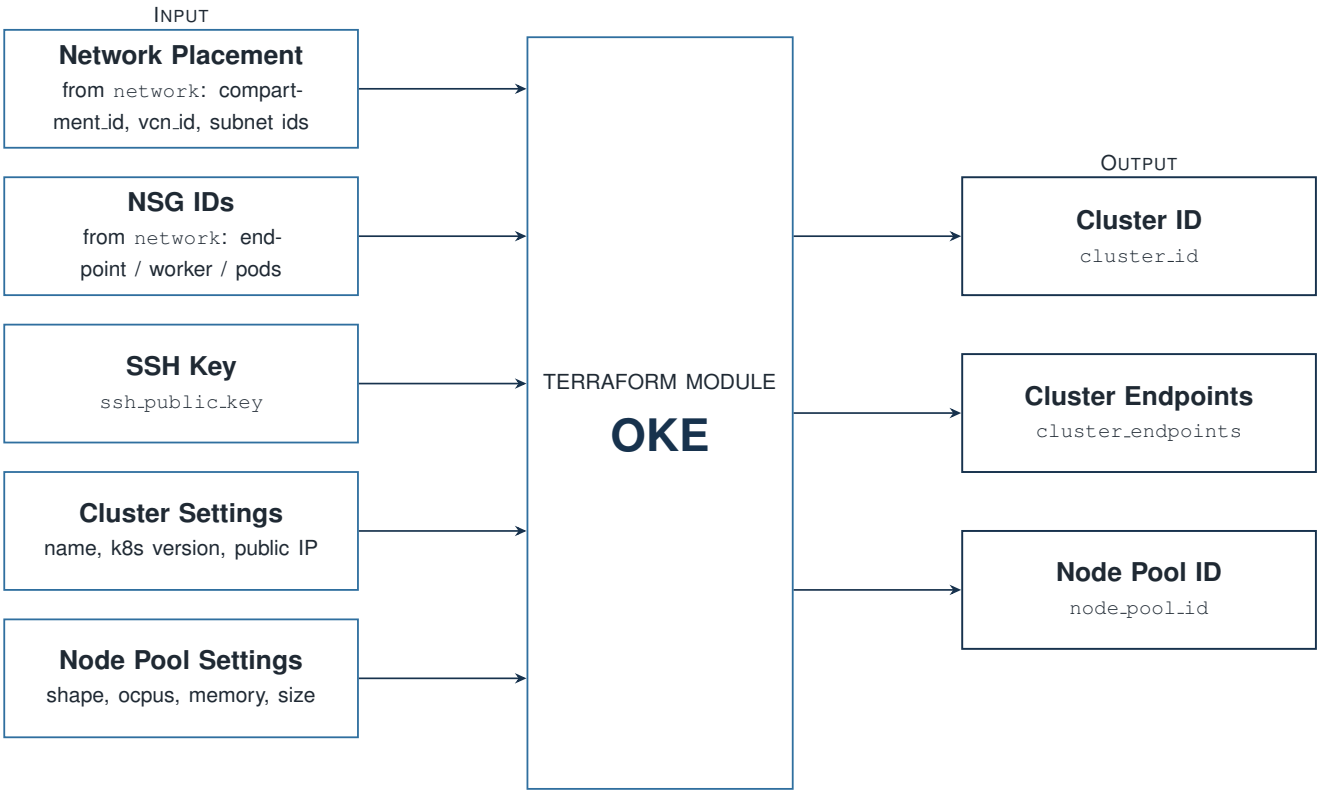

Figure 4.2: Terraform `oke` module inputs and outputs. Network placement and NSG IDs are consumed directly from the `network` module's outputs; the remaining inputs are root-level variables.

4.2 Root Module Configuration

We summarise the root Terraform module's variables in Table 4.1 (`variable.tf`).

Table 4.1: Root module variables.

Variable	Type	Purpose
`compartment_ocid`	string	Target compartment (required, no default)
`region`	string	OCI region (required, no default)
`node_shape`	string	OKE worker node compute shape, default `VM.Standard.A1.Flex`
`node_ocpus`	number	OCPUs per worker node, default `2`
`node_memory_in_gbs`	number	Memory per worker node, default `16`
`node_count`	number	Worker nodes per availability domain, default `2`, floored at `2` in `main.tf` regardless of input
`ssh_public_key`	string	SSH public key for worker node access
`ssh_allowed_cidr`	string	Declared but not currently read by any resource; SSH ingress is hard-coded to `0.0.0.0/0` in `modules/network` instead (Section 3.1)
`github_token`, `github_owner`, `github_repo`	string, sensitive	Used by `github.tf` to write the OKE cluster's OCID into the `Terraform-code` repository's own `CLUSTER_OCID` GitHub secret

4.3 Configuration Patterns

We declared every NSG ingress and egress rule, for every subnet, once as data in `modules/network/locals.tf` (`ingress_by_nsg`, `egress_by_nsg`), then flattened into one map keyed by `"<subnet>-<index>"` so a single `for_each` generates every `oci_core_network_security_group_security_rule` resource:

Listing 4.1: `modules/network/locals.tf` – flattening per-subnet rule lists into one for_each-able map

```
ingress_rules_flat = merge([
  for nsg_key, rules in local.ingress_by_nsg : {
    for idx, rule in rules :
    "${nsg_key}-${idx}" => merge(rule, { nsg_key = nsg_key })
  }
]...)
```

Therefore, adding a rule only requires one new object in `ingress_by_nsg` or `egress_by_nsg`; the resource block itself never changes. The same pattern builds the four subnets and their NSGs from typed maps rather than one block per subnet.

4.4 CI/CD Pipeline

We configured each of the five application repositories with its own `ci.yml` workflow, triggered on push to `main`. All five follow the same template: a multi-architecture image build (`linux/amd64`, `linux/arm64` via Docker Buildx and QEMU), OCIR authentication through a short-lived OCI CLI token, and a push tagged with both the GitHub Actions run number and `latest`. Moreover, the `arm64` target matches the OKE node pool itself: `node_shape` defaults to `VM.Standard.A1.Flex`, an Ampere ARM64 compute shape, so every image needs an ARM64 variant to run on the cluster's worker nodes. We summarise how a code change reaches the running cluster.

Table 4.2: CI/CD deployment sequence.

Step	What happens
Push to `main`	The service's own `ci.yml` builds and pushes a multi-arch image to OCIR
Reusable `update-helm.yml` workflow	Checks out `Terraform-code`, converts the service's dash-case name to `values.yaml`'s camelCase key (e.g. `payments-service` → `paymentsService`) via `awk`, and uses `yq` to bump only that key's `image.tag`
Commit and push to `Terraform-code`	The updated `values.yaml` is committed back with a bot identity (`GitHub Actions <actions@github.com>`)
ArgoCD reconciliation	ArgoCD's automated sync (`prune: true`, `selfHeal: true`) rolls out the new image for that one Deployment

We created Figure 4.3 to show this pipeline across all five repositories: each one builds its image and pushes it to OCIR, a shared checkout/update-charts/push job writes the new tag into the Helm chart, and ArgoCD watches that chart and updates the cluster. It runs automatically on every push and does not provision infrastructure, which is handled separately (Section 4.5).

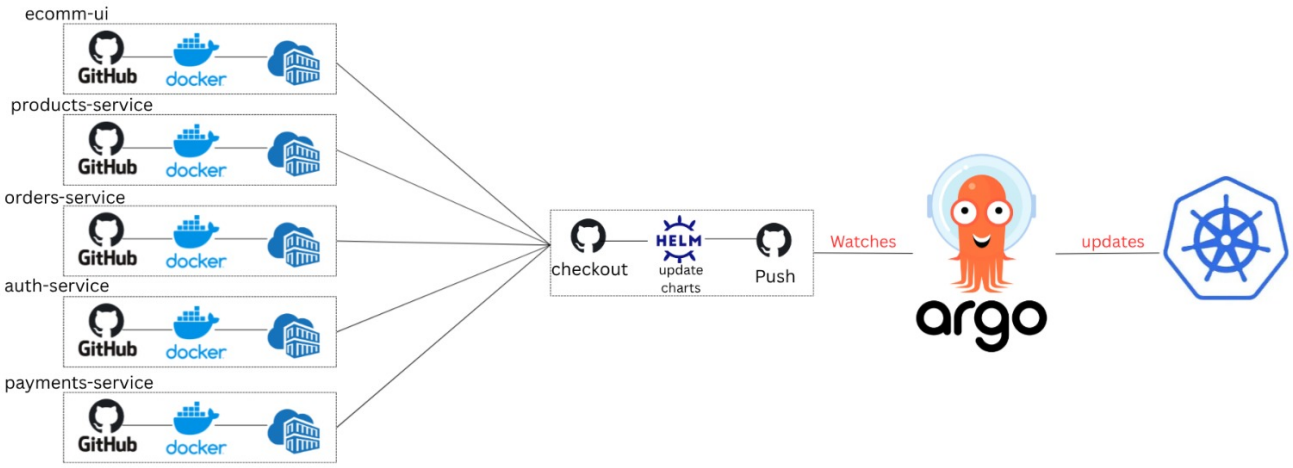

Figure 4.3: Image-promotion pipeline, from each service's CI workflow through to the ArgoCD-managed Kubernetes cluster.

4.5 Infrastructure Bootstrap Pipeline

We created `Terraform-code/.github/workflows/deploy.yml` to provision the environment from a clean account. Triggered manually (`workflow_dispatch`) rather than on push, it runs, in order: `terraform apply`; extracts the new cluster's OCID and load balancer subnet OCID from the Terraform output; generates a kubeconfig from that cluster OCID; installs the NGINX Ingress Controller via Helm, passing the load balancer subnet OCID through the `service.beta.kubernetes.io/oci-load-balancer-subnet1` annotation so OCI knows where to attach the Load Balancer; installs cert-manager and a `letsencrypt-prod` `ClusterIssuer` (HTTP-01, using the `ADMIN_EMAIL` secret as the ACME registration email) so the ingress's TLS certificate is issued automatically; creates the `ocir-secret` and `app-secrets` Kubernetes Secrets from encrypted GitHub Actions secrets; installs ArgoCD from its upstream manifest; and applies `argocd/ecommerce-app.yaml` and `argocd/argocd-ingress.yaml`. The kubeconfig step runs before any `helm`/`kubectl` command that depends on it. Together, these steps assemble the full stack: infrastructure, ingress controller and TLS, secrets, and the GitOps controller. Because Terraform state lives in the S3-compatible OCI backend rather than on the runner, re-running the workflow updates the existing infrastructure instead of provisioning a duplicate VCN or cluster.

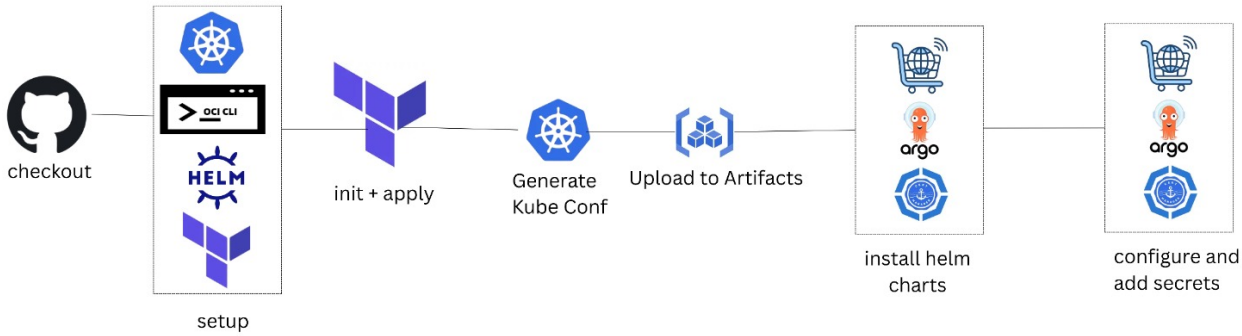

Figure 4.4: Infrastructure bootstrap pipeline: from checkout through Terraform provisioning, kubeconfig generation, Helm chart installation, and secret configuration.

We group the GitHub Actions secrets read by `deploy.yml` according to what they configure in Table 4.3.

Table 4.3: GitHub Actions secrets consumed by `deploy.yml`.

Secret	Used for
`OCI_TENANCY_OCID`, `OCI_USER_OCID`, `OCI_FINGERPRINT`, `OCI_KEY_CONTENT`	OCI CLI and Terraform provider authentication
`OCI_COMPARTMENT_OCID`	Target compartment for every provisioned resource
`SSH_PUBLIC_KEY`	Injected into the OKE node pool for worker node SSH access
`OCI_S3_ACCESS_KEY`, `OCI_S3_SECRET_KEY`, `OCI_NAMESPACE`	Terraform's S3-compatible remote state backend
`GH_PAT`	Used by `github.tf` to write the cluster OCID back to this repository, and by the reusable `update-helm.yml` workflow to commit image-tag bumps
`OCI_OCIR_USERNAME`, `OCI_AUTH_TOKEN`	Build the `ocir-secret` image-pull Secret
`JWT_SECRET`, `STRIPE_SECRET_KEY`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`	Build the `app-secrets` Kubernetes Secret; `ADMIN_EMAIL` also registers the Let's Encrypt `ClusterIssuer`

4.6 Container Images and Health Checks

We built every backend service from the same two-stage Dockerfile pattern, shown here for `auth-service`. We selected `node:20-slim` for all five images rather than an Alpine base: the OKE node pool runs ARM64, and cross-compiling Alpine images for `arm64` under QEMU produced `Illegal instruction` crashes at runtime, which the Debian-based slim image does not have.

Listing 4.2: `auth-service/Dockerfile`

```
FROM node:20-slim AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci || npm install
COPY . .

FROM node:20-slim AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --chown=node:node --from=builder /app /app
USER node
EXPOSE 5001

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:5001/health || exit 1

CMD ["npm", "start"]
```

For `ecomm-ui`, we used a three-stage variant (`deps`/`builder`/`runner`) that produces a Next.js standalone build, running as a dedicated `nextjs` user (UID 1001) rather than the base image's built-in `node` user. Both Dockerfile families run as non-root and expose a Docker-level `HEALTHCHECK`.

We summarise the Kubernetes liveness and readiness probe timing from the Helm chart's templates. The four backend services share identical timing; `ecomm-ui` uses longer, more forgiving timing against `/` rather than a dedicated health endpoint, to give its Next.js server-side rendering enough room to finish a cold start on the ARM64 node pool before Kubernetes restarts or deregisters the pod.

Table 4.4: Container health checks, by workload.

Workload	Liveness	Readiness
Backend services (4)	`GET /health`: 15s delay, every 10s, 5s timeout, 3 failures	`GET /health`: 5s delay, every 5s, 3s timeout, 3 failures
`ecomm-ui`	`GET /`: 40s delay, every 20s, 15s timeout, 5 failures	`GET /`: 30s delay, every 15s, 10s timeout, 5 failures

Moreover, the readiness probe's shorter delay and interval mean a pod is pulled out of rotation quickly if it stops responding, while the liveness probe's longer thresholds avoid restarting a container that is merely slow to start.

4.7 Deployment Guide

We based the steps below on `Terraform-code/DEPLOYMENT.md`; they cover the one-time setup of a new environment by hand; `deploy.yml` (Section 4.5) automates the same sequence. Once the environment

exists, ordinary code changes go through the per-push CI/CD path in Section 4.4 instead, with no manual step in between.

Prerequisites

Docker 20+, the OCI CLI, Terraform 1.5+, `kubectl` 1.28+, and Helm 3+, installed locally.

Deployment Steps

1. **Authenticate to OCIR** using a short-lived OCI CLI token, or a static OCI Auth Token as a fallback.
2. **Build and push all five images** with the repository's `scripts/build_and_push.sh <REGION> <NAMESPACE> <TAG> <COMPARTMENT>`.
3. **Provision infrastructure:**

```
          init
          plan
          apply
```

4. **Configure `kubectl`** via `oci ce cluster create-kubeconfig`, then apply the namespace:

```
kubectl apply -f ./kubernetes/00-namespace.yaml
```

5. **Create the OCIR pull secret and application secrets.** The pull secret is a Kubernetes `docker-registry` Secret built from an OCI Auth Token; application secrets come from a local, gitignored `.env` file:

```
kubectl create secret docker-registry ocir-secret \
  --docker-server=jed.ocir.io \
  --docker-username='<OCIR username>' \
  --docker-password='<OCI auth token>' \
  -n ecommerce-ns

kubectl create secret generic app-secrets \
  --    -env-file=.env -n ecommerce-ns
```

6. **Deploy the application** – either the Helm chart (the path ArgoCD also reconciles against) or the raw manifests:

```
helm install ecommerce ./helm/ecommerce-app -n ecommerce-ns
% or, applying each raw manifest in numeric order (00 through 06)
```

7. **Register the Application with ArgoCD** by applying `argocd/ecommerce-app.yaml`, so future pushes flow through the CI/CD pipeline instead of manual commands.

4.8 Configuration Reference

We summarise the environment variables the application reads at runtime.

Table 4.5: Application environment variables.

Variable	Read by
MONGODB_URI	All four backend services; also referenced by ecomm-ui's unused src/lib/db.ts helper
JWT_SECRET	auth-service (sign) and ecomm-ui's middleware.ts (verify)
ADMIN_EMAIL, ADMIN_PASSWORD	auth-service, used only to seed the default admin account when none exists
STRIPE_SECRET_KEY	payments-service
AUTH_SERVICE_URL, PRODUCT_SERVICE_URL, ORDER_SERVICE_URL, PAYMENT_SERVICE_URL	ecomm-ui's BFF routes, to reach each backend service by its in-cluster Service DNS name
PORT	Every service; defaults to its own hard-coded port if unset

We source the four *_SERVICE_URL variables and MONGODB_URI from a ConfigMap (app-config); however, the remaining values come from the app-secrets Secret described in Section 4.5.

4.9 Testing

To strengthen quality assurance across our project, we created the dedicated `EjadaStore-Testing` repository. We used it to run automated API, security, and regression tests against our production deployment at `https://shoy.publicvm.com`. During our initial QA cycle, we combined live endpoint validation with source-code review and identified 15 opportunities for improvement: six critical, five high, and four medium priority.

Critical Remediation Results

We use Table 4.6 to record the six critical cases and the result of the production retest after the fixes were deployed.

Table 4.6: Critical QA findings verified after remediation.

ID	Verified behavior after remediation	Retest status
`BUG-001`	Anonymous product-creation requests are rejected.	Passed
`BUG-003`	Orders with zero or negative item quantities are rejected before stock is changed.	Passed
`BUG-006`	An unauthenticated caller cannot register a new administrator account.	Passed
`BUG-008`	Anonymous order-status updates are rejected.	Passed
`BUG-009`	The production database seed and wipe endpoint is protected from unauthenticated use.	Passed
`BUG-018`	Fabricated Stripe Checkout Session IDs are rejected.	Passed

Team Response to the Payment-Service Regression

During remediation, we found that overlapping changes in the payments service temporarily removed `server.ts` and its associated schema, which prevented the service from running correctly. We identified the regression during coordination and retesting. Ali Hamad restored the deleted payment-service files and corrected the CORS allowlist, while Abdelrahman Darwish reconciled the server and schema changes and pushed the consolidated update. Afterward, we tested the deployed service again as part of the full production run. This recovery is recorded as a team remediation activity rather than as a separate product defect because it occurred while the original findings were being fixed.

Final Production Retest

In our follow-up run, we executed 95 checks and passed 93, giving us a rounded pass rate of 98%. We confirmed that all six critical security cases passed and that the corrected CORS rules were active in the deployed services. Two data-integrity checks still failed, associated with `BUG-010` and `BUG-011`: legacy database records still contained a null category reference and a negative price created before the corresponding fixes. These results helped us distinguish application remediation from historical data cleanup: we corrected the code path, while old records still required migration or reseeding.

Testing Outcome

After our fixes and redeployment, 93 of 95 production checks passed. We closed the six critical findings through retesting; the remaining two failures concerned legacy database contents, not a recurrence of the payment-service regression.

We currently maintain this QA suite separately from the six implementation repositories. `Terraform-code/.github/workflows/test-runner.yml` still contains only a placeholder runner check, so integrating the regression suite into CI is a future improvement.

4.10 Environment Teardown

For teardown, we follow the dependency order documented in `DEPLOYMENT.md`, uninstalling the application before destroying the infrastructure that hosts it:

Listing 4.3: Teardown sequence from DEPLOYMENT.md

```
helm uninstall ecommerce -n ecommerce-ns
kubectl delete -f ./kubernetes/

          destroy
```

5 System Components and Runtime Behavior

In this chapter, we summarise what we observed in the deployed workloads, request routing, resource allocation, frontend implementation, and reviewed repository state.

5.1 Component Reference

We summarise the deployed workloads, their OCIR repositories, and their ports.

Table 5.1: Deployed workloads and ports.

Component	OCIR repository	Port
`ecomm-ui`	`shared-group-a-cmp/ecomm-ui`	container 3000, Service 80 (LoadBalancer)
`auth-service`	`shared-group-a-cmp/auth-service`	5001
`products-service`	`shared-group-a-cmp/products-service`	5002
`orders-service`	`shared-group-a-cmp/orders-service`	5003
`payments-service`	`shared-group-a-cmp/payments-service`	5004
`mongodb`	`docker.io/library/mongo:7.0` (not OCIR)	27017

Moreover, we observed that `values.yaml` records image tags 17 (auth), 11 (products), 9 (orders), 8 (payments), and 11 (ecomm-ui), reflecting multiple runs of the GitOps pipeline.

5.2 Request Routing

We use Table 5.2 to show the path-based routing defined in the ingress resource.

Table 5.2: Ingress routing rules.

Path	Routed to	Port
`/`	`ecomm-ui`	80
`/api/auth`	`auth-service`	5001
`/api/products`	`products-service`	5002
`/api/orders`	`orders-service`	5003
`/api/payments`	`payments-service`	5004

We configured the ingress host as `shoy.publicvm.com`, with a TLS block referencing a `shoy-tls-secret`. That certificate is provisioned automatically: the bootstrap workflow installs cert-manager and a `letsencrypt-prod ClusterIssuer` using the HTTP-01 solver through the NGINX ingress class, so `shoy-tls-secret` is issued by Let's Encrypt rather than created by hand (Section 4.5).

5.3 Scaling and Resource Allocation

Autoscaling Configuration

We found that the Helm chart (`helm/ecommerce-app/templates/hpa.yaml`), which ArgoCD reconciles, scales every workload on CPU utilization only, at 75%. The raw manifest (`kubernetes/06-hpa.yaml`) scales the same workloads on both CPU (75%) and memory (80%). Table 5.3 shows the Helm policy, since that is the one GitOps keeps live.

Table 5.3: HPA policy (Helm chart), uniform across all five workloads.

Min replicas	Max replicas	Scaling target
2	5	75% CPU (Helm) – or 75% CPU + 80% memory (raw manifest only)

Moreover, we summarise the per-pod CPU and memory requests and limits in Table 5.4; the four backend services use identical values.

Table 5.4: Per-pod resource requests and limits.

Workload	CPU request / limit	Memory request / limit
`auth-service`, `products-service`, `orders-service`, `payments-service`	100m / 500m	128Mi / 256Mi
`ecomm-ui`	200m / 1000m	256Mi / 512Mi
`mongodb`	200m / 1000m	512Mi / 1Gi

5.4 Frontend Implementation Status

We summarise the implementation status we observed in the storefront and administration pages in Table 5.5.

Table 5.5: Frontend implementation status.

Page / feature	Status
Homepage (`/`)	Working – fetches from `products-service`, renders the first 10 products
Cart (`/cart`), checkout (`/checkout`)	Working – cart is client-side state (React Context + `localStorage`); checkout creates a Stripe Checkout Session
Admin products / orders management	Working – full CRUD via Next.js Server Actions and BFF routes calling the backend services directly
All-products listing (`/products`)	Incomplete – displays placeholder content; the fetch and rendering logic is sketched out in code comments
Product detail (`/products/[id]`)	Incomplete – same pattern; there is currently no way to view a single product's detail page or add to cart from it
Categories (`/categories`)	Incomplete – same pattern
Admin dashboard KPIs (`/admin`)	Displays static placeholder metrics (e.g. "$45,231.89" revenue, "356" orders) rather than data computed from the running system
User login/registration	Credentials are validated server-side; no session or token is issued afterward (Section 3.2)

Moreover, we found a second cart implementation using Zustand (`src/hooks/useCart.ts`); it is imported by one component (`components/modules/products/ProductCard.tsx`) that is not rendered by any page currently in use. The cart implementation used by the working pages is `src/context/CartContext.tsx`.

5.5 Sources Reviewed

We based this documentation on our review of all six `Ejada-A` repositories: application source, Terraform configuration, Helm charts, Kubernetes manifests, and CI/CD workflows. In Table 5.6, we record the commit at which we reviewed each repository.

Table 5.6: Repositories and commits this documentation reflects.

Repository	Commit
`auth-service`	`f6a4ca1`
`products-service`	`b45b8cb`
`orders-service`	`f63ecd4`
`payments-service`	`83cd162`
`ecomm-ui`	`197e0e7`
`Terraform-code`	`ebfdc17`

6 Limitations and Future Improvements

Current Limitations

Our review identified a few areas that are still early or incomplete. Although we closed the six critical QA findings during the production remediation cycle (Section 4.9), we found that regular users are still not issued a session or token. Moreover, the `/products`, `/products/[id]`, and `/categories` pages are still placeholders (Section 5.4), as is the admin dashboard's metrics display. We also found that MongoDB runs without authentication or redundancy (Section 3.6), and `values.yaml` still carries an unused plaintext secrets block worth cleaning up (Section 3.4). In addition, the Kubernetes API endpoint, worker SSH, and the NodePort range are currently reachable from `0.0.0.0/0` (Section 3.1), and the Helm chart and raw Kubernetes manifests define slightly different HPA policies (Section 5.3). The production regression suite is maintained separately rather than executed by our six repositories' CI workflows, and a few frontend dependencies (`zod`, `next-auth`, `bcryptjs` in `ecomm-ui`) are declared but not yet used.

6.1 Our Recommended Next Steps

Based on these findings, we recommend the following next steps for continuing our work:

1. **Integrate the production regression suite into CI** so the 95 checks run automatically before deployment and prevent a recurrence of the payment-service file regression.
2. **Complete the `/products`, `/products/[id]`, and `/categories` pages**; the fetch and rendering logic is already sketched out in code comments in each file.
3. **Remove the plaintext secrets block** in `values.yaml` and rotate the associated credentials.
4. **Restrict the currently-public network paths**: scope the Kubernetes API endpoint and worker SSH access to known IPs instead of `0.0.0.0/0`, and disable the direct NodePort path now that the Ingress Controller is in place.
5. **Enable MongoDB authentication** and consider whether the five workloads should keep sharing one instance and one set of collections.

7 Troubleshooting

Based on our deployment and testing work, we assembled Table 7.1 to connect common symptoms with diagnostic commands and likely causes. We based these checks on the operational commands in `Terraform-code/DEPLOYMENT.md` and the deployment mechanics in Chapter 4.

Table 7.1: Common symptoms, diagnostic commands, and likely causes.

Symptom	Diagnostic command	Likely cause
A pod is stuck in `CrashLoopBackOff`	`kubectl logs <pod> -n ecommerce-ns --previous`	Missing or misconfigured secret (Table 4.5), or the service cannot reach MongoDB
A pod never becomes `Ready`	`kubectl describe pod <pod> -n ecommerce-ns`	Failing readiness probe against `/health`; check the probe timing in Table 4.4
A new image never rolls out after a push	Check the run status of `ci.yml` and `update-helm.yml` in the service's repository, then `git log -p values.yaml` in `Terraform-code`	The reusable `update-helm.yml` workflow failed to bump the tag, or ArgoCD has not yet synced
ArgoCD shows `OutOfSync`	`argocd app get ecommerce-app`	`selfHeal` will revert any manual `kubectl edit`; the fix is always a Git commit to `Terraform-code`, never a live edit
A request to a backend path returns 502/504	`kubectl get endpoints -n ecommerce-ns` and re-check Table 5.2	The target Service has no Ready pods behind it, or the ingress path/port does not match the Service definition
Pods across the namespace show `ImagePullBackOff` at the same time	`kubectl describe pod <pod> -n ecommerce-ns` and check the `ocir-secret` age	The `OCI_AUTH_TOKEN` behind `ocir-secret` is a permanent OCI Auth Token; if it was ever regenerated as a short-lived CLI token instead, pulls fail once it expires
An order shows `paid` that was never actually paid	`kubectl logs -n ecommerce-ns -l app=payments-service --tail=50` and check the Stripe dashboard for a matching session	Stripe session validation or order-to-session matching may have regressed; compare the request with the verified behavior in Section 4.9

A Glossary

Table A.1: Terms used throughout this documentation.

Term	Meaning
OKE	Oracle Kubernetes Engine: OCI's managed Kubernetes service.
OCIR	Oracle Cloud Infrastructure Registry: the container image registry each service pushes to.
NSG	Network Security Group: a firewall attached to individual resources rather than a whole subnet.
HPA	Horizontal Pod Autoscaler: scales a Deployment's replica count between a minimum and maximum based on resource utilization.
BFF	Backend for Frontend: here, `ecomm-ui`'s own `/api/*` routes, which proxy to the corresponding backend service over HTTP. This is the only inter-service orchestration in the system – the backend services never call each other.
GitOps	A deployment model where the desired state of the cluster lives in a Git repository, and an operator (ArgoCD) continuously reconciles the live cluster to match it.
Helm chart	A templated bundle of Kubernetes manifests, parameterized by a single `values.yaml` file.
ArgoCD	The GitOps controller that watches the Helm chart in `Terraform-code` and applies it to the cluster.
PVC	Persistent Volume Claim: a request for durable storage, here backing the shared MongoDB deployment.
ClusterIP	A Kubernetes Service type reachable only from inside the cluster, used for internal service-to-service calls.
kube-proxy	The per-node Kubernetes component that implements Service-to-Pod routing; the Load Balancer's health check queries it directly on each worker node.
VCN	Virtual Cloud Network: OCI's software-defined network, the container for every subnet, gateway, and NSG in this design.
NAT Gateway	Provides outbound-only internet access for the private Worker Nodes and Pods subnets.
Service Gateway	Provides a private path from the private subnets to other OCI services (e.g. OCIR) without transiting the NAT Gateway.

B Network Configuration Reference

In this reference, we document how we attached every ingress and egress rule at the resource level through a Network Security Group rather than a subnet-wide security list. Tables B.3 through B.6 list the rules declared in `modules/network/locals.tf` for each of the four subnets (Section 2.5). Rows marked *optional, enabled* come from a boolean toggle that defaults to `false` in the network module but is set to `true` in the root module (`main.tf`); see Section 3.1.

Figure B.1: OCI Console Network Visualizer view of `groupa-dev-vcn`, showing the Internet Gateway (`groupa-dev-igw`), NAT Gateway (`groupa-dev-nat`), and Service Gateway (`groupa-dev-svc-...`) attached to the VCN, and the default security list shared by all four subnets.

Table B.1: Gateways attached to the VCN.

Gateway	Attached to	Purpose
Internet Gateway	API Endpoint, Load Balancer subnets	Inbound/outbound internet access for public subnets
NAT Gateway	Worker Nodes, Pods subnets	Outbound-only internet access for private subnets
Service Gateway	Worker Nodes, Pods subnets	Private, fast path to OCI services (e.g. OCIR) without using NAT

Table B.2: Route tables for each subnet.

Subnet	Destination	Target
API Endpoint (public)	0.0.0.0/0	Internet Gateway
Load Balancer (public)	0.0.0.0/0	Internet Gateway
Worker Nodes (private)	All region services in Oracle Services Network	Service Gateway
Worker Nodes (private)	0.0.0.0/0	NAT Gateway
Pods (private)	All region services in Oracle Services Network	Service Gateway
Pods (private)	0.0.0.0/0	NAT Gateway

Table B.3: NSG rules for the Kubernetes API Endpoint subnet.

Direction	Peer	Port	Description
Ingress	**0.0.0.0/0**	TCP 6443	External access to the Kubernetes API endpoint – not restricted to team IPs
Ingress	Worker Nodes CIDR	TCP 6443, 12250	Worker to API endpoint communication
Ingress	Pods CIDR	TCP 6443, 12250	Pod to API endpoint communication (VCN-native)
Ingress	Worker Nodes CIDR	ICMP 3,4	Path MTU Discovery
Egress	OCI Services Network	TCP 443	API endpoint to OKE communication
Egress	Pods CIDR	ALL	API endpoint to pod communication (VCN-native)
Egress	Worker Nodes CIDR	ICMP 3,4 (two rules)	Path MTU Discovery
Egress	Worker Nodes CIDR	TCP 10250	API endpoint to worker node communication

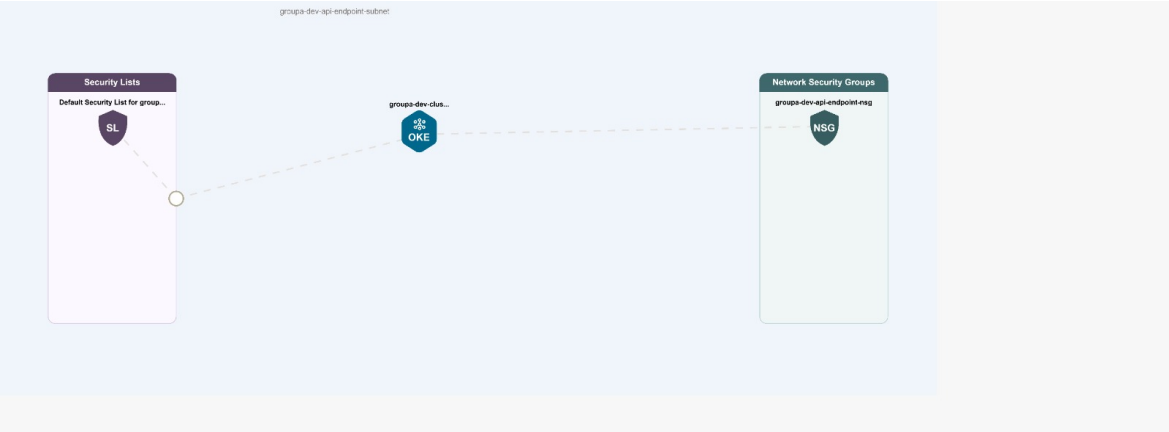

Figure B.2: Network Visualizer view of the `groupa-dev-api-endpoint-subnet`, showing the OKE cluster's Kubernetes API endpoint attached to the `groupa-dev-api-endpoint-nsg`.

Table B.4: NSG rules for the Worker Nodes subnet.

Direction	Peer	Port	Description
Ingress	Worker Nodes CIDR	ALL	Communication between worker nodes
Ingress	Pods CIDR	ALL	Pods on one node reach pods on other nodes
Ingress	API Endpoint CIDR	TCP ALL	API endpoint to worker node communication
Ingress	0.0.0.0/0	ICMP 3,4	Path MTU Discovery
Ingress	Load Balancer CIDR	TCP 10256	LB to kube-proxy health check on worker nodes
Ingress	**0.0.0.0/0** (optional, enabled)	TCP 22	SSH access for troubleshooting
Ingress	**0.0.0.0/0** (optional, enabled)	TCP 30000–32767	Direct NodePort traffic, bypassing the Load Balancer
Egress	Worker Nodes CIDR	ALL	Communication between worker nodes
Egress	Pods CIDR	ALL	Worker nodes reach pods on other nodes
Egress	0.0.0.0/0	ICMP 3,4	Path MTU Discovery
Egress	OCI Services Network	TCP ALL	Nodes communicate with OKE
Egress	API Endpoint CIDR	TCP 6443, 12250	Worker to API endpoint communication
Egress	**0.0.0.0/0** (optional, enabled)	TCP ALL	General outbound internet access

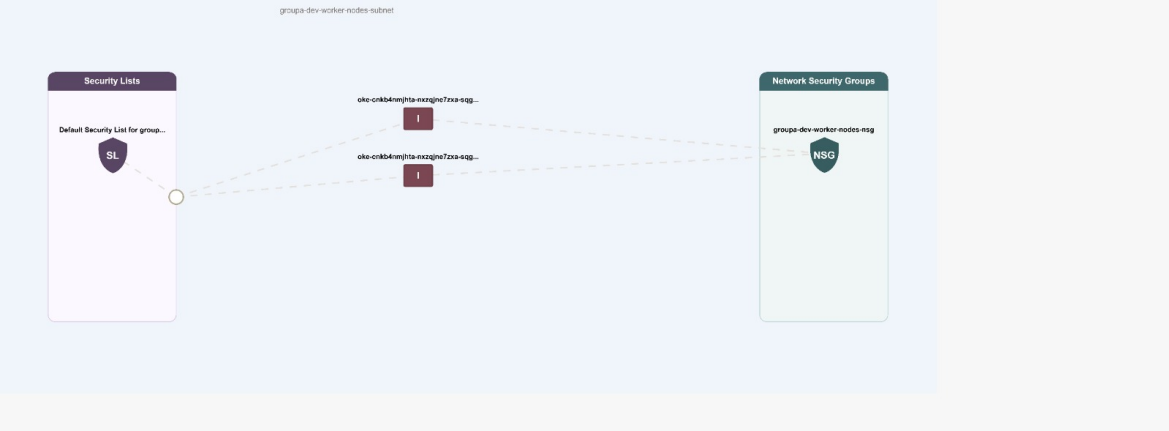

Figure B.3: Network Visualizer view of the `groupa-dev-worker-nodes-subnet`, showing the two OKE-managed ingress rules attached to the `groupa-dev-worker-nodes-nsg`.

Table B.5: NSG rules for the Pods subnet.

Direction	Peer	Port	Description
Ingress	API Endpoint CIDR	ALL	API endpoint to pod communication
Ingress	Worker Nodes CIDR	ALL	Pods on one node reach pods on other nodes
Ingress	Pods CIDR	ALL	Pods communicate with each other
Egress	Pods CIDR	ALL	Pods communicate with each other
Egress	OCI Services Network	ICMP 3,4	Path MTU Discovery
Egress	OCI Services Network	TCP ALL	Pods communicate with OCI services
Egress	API Endpoint CIDR	TCP 6443, 12250	Pod to API endpoint communication
Egress	**0.0.0.0/0** (optional, enabled)	TCP 443	Outbound internet access

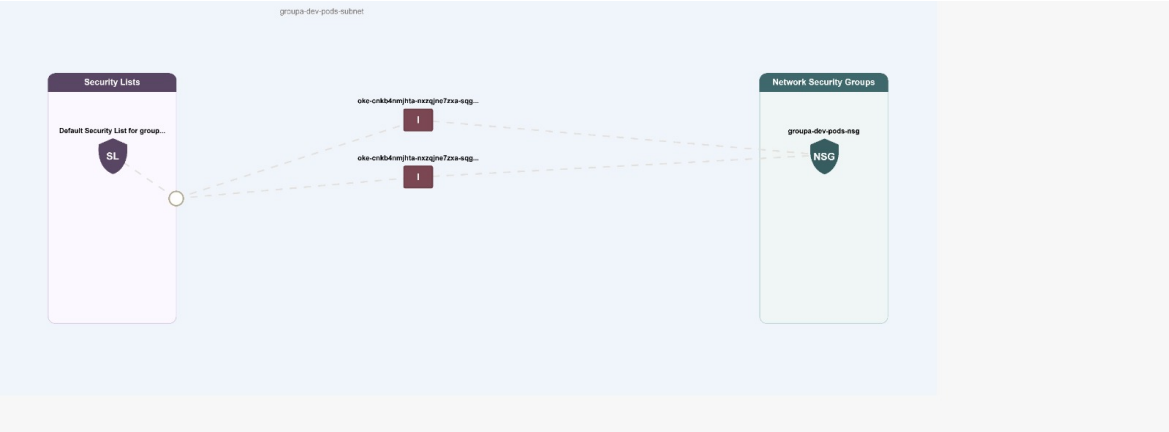

Figure B.4: Network Visualizer view of the `groupa-dev-pods-subnet`, showing the same OKE-managed ingress rules attached to the `groupa-dev-pods-nsg`.

Table B.6: NSG rules for the Load Balancer subnet.

Direction	Peer	Port	Description
Ingress	0.0.0.0/0	TCP 80	Public HTTP traffic to the Load Balancer
Ingress	0.0.0.0/0	TCP 443	Public HTTPS traffic to the Load Balancer
Egress	Worker Nodes CIDR	TCP 30000–32767	LB forwards traffic to worker nodes
Egress	Worker Nodes CIDR	TCP 10256	LB communicates with kube-proxy health check

Figure B.5: Network Visualizer view of the `groupa-dev-load-balancer-subnet`, showing the public Load Balancer attached to the `groupa-dev-load-balancer-nsg`.

We use *optional, enabled* to mark rules that come from boolean switches in `modules/network/variables.tf` (SSH access, general worker outbound, pod outbound internet, and public NodePort access) that default to `false`; the root module sets all four to `true`. See Section 3.1 for the security implications.

C Command Reference

Table C.1: Commands used to deploy, operate, and tear down the system, from `Terraform-code/DEPLOYMENT.md`.

Command	Used for
`terraform init / plan / apply`	Provision the VCN, subnets, OKE cluster, and OCIR repositories
`oci ce cluster create-kubeconfig`	Generate a local `kubectl` context for the OKE cluster
`./scripts/build_and_push.sh <region> <namespace> <tag> <compartment>`	Build and push all five images to OCIR in one step
`kubectl create secret docker-registry ocir-secret ...`	Create the image-pull Secret from an OCI Auth Token
`kubectl create secret generic app-secrets --from-env-file=.env`	Create the application Secret from a local, gitignored `.env` file
`kubectl apply -f ./kubernetes/`	Apply the raw Kubernetes manifests directly (the non-GitOps deployment path)
`helm install ecommerce ./helm/ecommerce-app -n ecommerce-ns`	Deploy the application through the Helm chart (the GitOps-managed path)
`kubectl rollout restart deployment -n ecommerce-ns`	Restart all microservice deployments in the namespace
`helm uninstall ecommerce -n ecommerce-ns`	Remove the Helm-managed application before infrastructure teardown
`terraform destroy`	Tear down the OKE cluster and network once the application layer is removed